llnl_c1/lulesh · lulesh.cc · llm_pragmas_v5 · claude-haiku-4-5-20251001 · BROKEN

−53 / +326 lines of 1967 · 31 parallel region(s) added · 26 of 80 candidate(s) kept · RESTRUCTURING (232 code line(s) added, 44 removed) · restructuring level 0

BEFORE

```
459                                 Real_t *x8n, Real_t *y8n, Real_t *z8n,
460                                 Real_t *dvdx, Real_t *dvdy, Real_t *dvdz,
461                                 Real_t hourg, Index_t numElem,
462                                 Index_t numNode)
463   {
464
465      Real_t  gamma[4][8];
466
467      gamma[0][0] = Real_t( 1.);
468      gamma[0][1] = Real_t( 1.);
469      gamma[0][2] = Real_t(-1.);
      …
495      gamma[3][4] = Real_t( 1.);
496      gamma[3][5] = Real_t(-1.);
497      gamma[3][6] = Real_t( 1.);
498      gamma[3][7] = Real_t(-1.);
499
500      for(Index_t i2=0;i2<numElem;++i2){
501         Real_t hgfx[8], hgfy[8], hgfz[8] ;
502
503         Real_t coefficient;
504
      …
607
608         CalcElemFBHourglassForce(xd1,yd1,zd1,
609                         hourgam,
610                         coefficient, hgfx, hgfy, hgfz);
611
612         domain.fx(n0si2) += hgfx[0];
613         domain.fy(n0si2) += hgfy[0];
614         domain.fz(n0si2) += hgfz[0];
615
616         domain.fx(n1si2) += hgfx[1];
617         domain.fy(n1si2) += hgfy[1];
618         domain.fz(n1si2) += hgfz[1];
619
620         domain.fx(n2si2) += hgfx[2];
621         domain.fy(n2si2) += hgfy[2];
622         domain.fz(n2si2) += hgfz[2];
623
624         domain.fx(n3si2) += hgfx[3];
625         domain.fy(n3si2) += hgfy[3];
626         domain.fz(n3si2) += hgfz[3];
627
628         domain.fx(n4si2) += hgfx[4];
629         domain.fy(n4si2) += hgfy[4];
630         domain.fz(n4si2) += hgfz[4];
631
632         domain.fx(n5si2) += hgfx[5];
633         domain.fy(n5si2) += hgfy[5];
634         domain.fz(n5si2) += hgfz[5];
635
636         domain.fx(n6si2) += hgfx[6];
637         domain.fy(n6si2) += hgfy[6];
638         domain.fz(n6si2) += hgfz[6];
639
640         domain.fx(n7si2) += hgfx[7];
641         domain.fy(n7si2) += hgfy[7];
642         domain.fz(n7si2) += hgfz[7];
643
644      }
645
646   }
647
648   static inline
649   void CalcHourglassControlForElems(Domain& domain,
      …
656      Real_t *dvdz = Allocate<Real_t>(numElem8) ;
657      Real_t *x8n  = Allocate<Real_t>(numElem8) ;
658      Real_t *y8n  = Allocate<Real_t>(numElem8) ;
```

AFTER

```
459                                 Real_t *x8n, Real_t *y8n, Real_t *z8n,
460                                 Real_t *dvdx, Real_t *dvdy, Real_t *dvdz,
461                                 Real_t hourg, Index_t numElem,
462                                 Index_t numNode)
463   {
464      Real_t  gamma[4][8];
465
466      gamma[0][0] = Real_t( 1.);
467      gamma[0][1] = Real_t( 1.);
468      gamma[0][2] = Real_t(-1.);
      …
494      gamma[3][4] = Real_t( 1.);
495      gamma[3][5] = Real_t(-1.);
496      gamma[3][6] = Real_t( 1.);
497      gamma[3][7] = Real_t(-1.);
498
499      /* Allocate per-element buffers to store computed hourglass forces (flat ar
500      Real_t *hgfx_elem = Allocate<Real_t>(numElem * 8);
501      Real_t *hgfy_elem = Allocate<Real_t>(numElem * 8);
502      Real_t *hgfz_elem = Allocate<Real_t>(numElem * 8);
503
504      #pragma omp parallel for firstprivate(hourg, numElem)
505      for(Index_t i2=0;i2<numElem;++i2){
506         Real_t hgfx[8], hgfy[8], hgfz[8] ;
507
508         Real_t coefficient;
509
      …
612
613         CalcElemFBHourglassForce(xd1,yd1,zd1,
614                         hourgam,
615                         coefficient, hgfx, hgfy, hgfz);
616
617         /* Store computed forces into per-element buffers (each thread writes to
618         for(Index_t j=0; j<8; ++j) {
619            hgfx_elem[i2*8 + j] = hgfx[j];
620            hgfy_elem[i2*8 + j] = hgfy[j];
621            hgfz_elem[i2*8 + j] = hgfz[j];
622         }
623
624      }
625
626      /* Apply all accumulated hourglass forces to domain nodes (serial or with m
627      #pragma omp critical(hg_force_apply)
628      {
629         for(Index_t i2=0; i2<numElem; ++i2) {
630            const Index_t *nodeList = domain.nodelist(i2);
631            for(Index_t j=0; j<8; ++j) {
632               Index_t node = nodeList[j];
633               domain.fx(node) += hgfx_elem[i2*8 + j];
634               domain.fy(node) += hgfy_elem[i2*8 + j];
635               domain.fz(node) += hgfz_elem[i2*8 + j];
636            }
637         }
638      }
639
640      /* Clean up temporary buffers */
641      Release(&hgfz_elem);
642      Release(&hgfy_elem);
643      Release(&hgfx_elem);
644
645   }
646
647   static inline
648   void CalcHourglassControlForElems(Domain& domain,
      …
655      Real_t *dvdz = Allocate<Real_t>(numElem8) ;
656      Real_t *x8n  = Allocate<Real_t>(numElem8) ;
657      Real_t *y8n  = Allocate<Real_t>(numElem8) ;
```

file

red = removed green = added yellow = parallel region (#pragma omp) right strip = the whole file: green where changed, yellow where a parallel region now is